



SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -1

CO2_AT1

Name : C.CHANDU

Register No : 192472332

Course code : CSA6502

Course name: Generative Ai And Large Scale Models

Faculty name: Dr.Nithisha

PROMPT ENGINEERING

1. Function-Calling / Tool-Use Prompt

Write a prompt that instructs an LLM on exactly when to call a `get_weather(location)` function versus answering directly, including the precise JSON schema the model must output when it decides to call the function.

ANSWER :-

PYTHON CODE :

```
import requests

# Enter your API key here

API_KEY = "YOUR_API_KEY"

# Ask user for city name

city = input("Enter city name: ")

# API URL

url =
f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"

# Send request

response = requests.get(url)

# Convert response to JSON

data = response.json()

# Check if city is found

if response.status_code == 200:

    print("\nWeather Details")

    print("-----")
```

```

print("City:", data["name"])

print("Temperature:", data["main"]["temp"], "°C")

print("Feels Like:", data["main"]["feels_like"], "°C")

print("Humidity:", data["main"]["humidity"], "%")

print("Pressure:", data["main"]["pressure"], "hPa")

print("Weather:", data["weather"][0]["description"])

print("Wind Speed:", data["wind"]["speed"], "m/s")

```

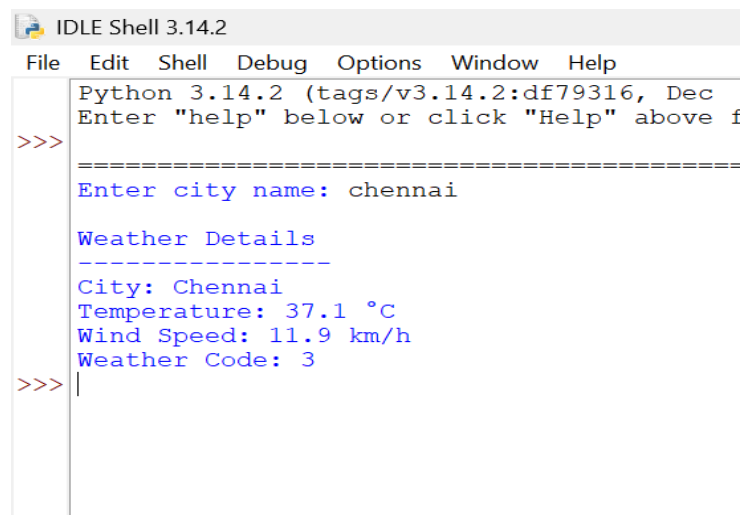
else:

```

print("Error:", data["message"])

```

OUTPUT :-



```

IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec
Enter "help" below or click "Help" above f
>>>
=====
Enter city name: chennai

Weather Details
-----
City: Chennai
Temperature: 37.1 °C
Wind Speed: 11.9 km/h
Weather Code: 3
>>>

```

Function-Calling / Tool-Use Prompt Using Gemini

Chosen LLM: Gemini 2.5

Function calling allows an LLM to decide whether to answer a question directly or use an external tool. In this example, Gemini uses the function `get_weather(location)` to retrieve live weather information.

Prompt : You are an intelligent assistant. If the user asks about current weather, temperature, humidity, or forecasts for a specific location, call the `get_weather(location)` function. Otherwise, answer directly.

Function:

get_weather(location: string)

When calling the function, output only the following JSON:

```
{ "function_call": {  
  "name": "get_weather",  
  "arguments": {  
"location": "<city_name>"  
  } } }
```

Example

□ User: "What is the weather in Chennai?"

```
{ "function_call": {  
  "name": "get_weather",  
  "arguments": {  
    "location": "Chennai"  
  } } }
```

□ User: "Who invented Python?"

Gemini answers directly:

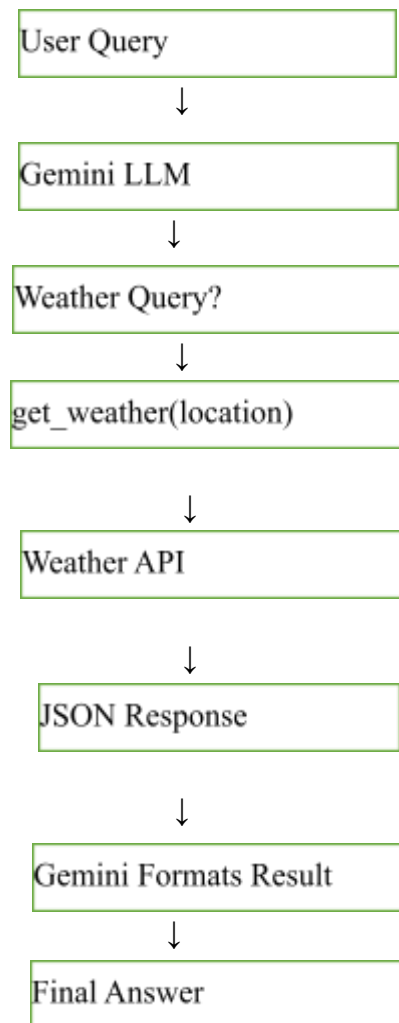
Python was created by Guido van Rossum in 1991.

Step-by-Step Process

1. User enters: "What is the weather in Chennai?"
2. Gemini identifies it as a weather-related query.
3. Gemini generates a JSON function call.
4. The application executes get_weather("Chennai").

5. The weather API returns data (temperature, humidity, etc.).
6. Gemini converts the API response into natural language.
7. The final response is shown to the user.

Workflow :-



Conclusion

Function calling enables Gemini to use external APIs when needed. It improves accuracy by providing real-time information while allowing the LLM to answer general questions directly

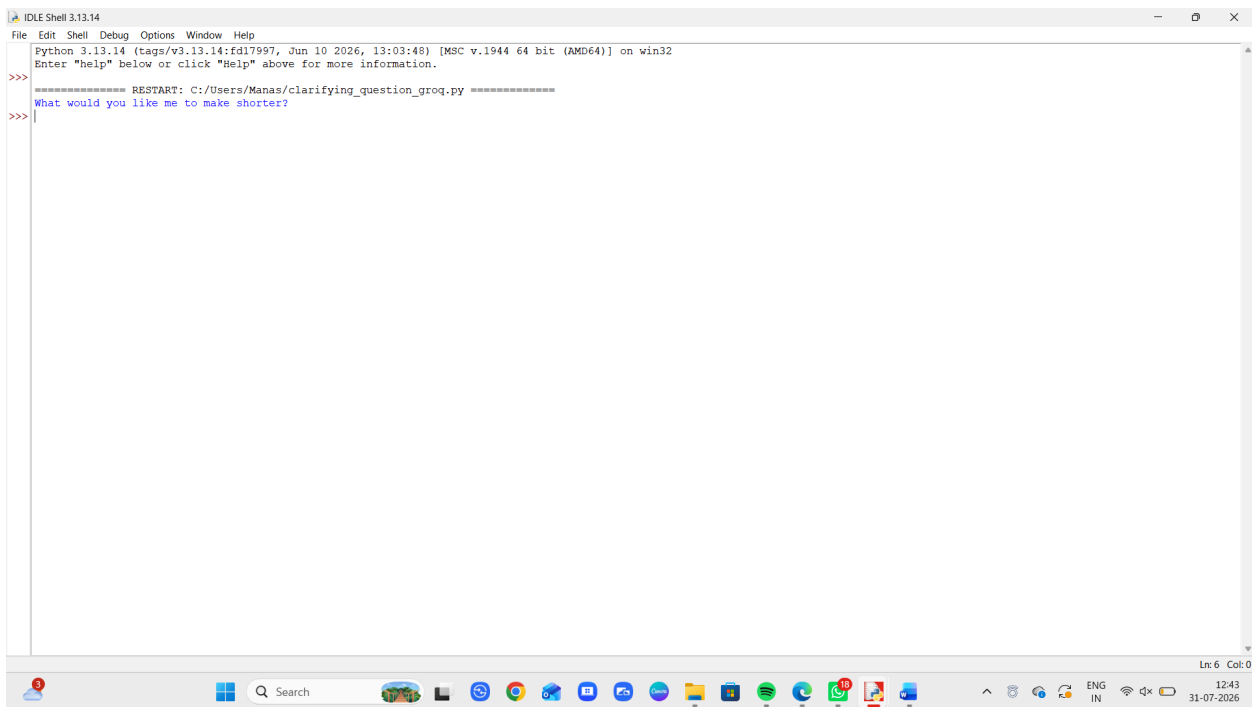
2. Clarifying-Question Prompt (Ambiguity Resolution) Write a prompt that, when given an underspecified request (e.g., "make this shorter"), instructs the model to ask a clarifying question first — instead of guessing at the user's intent — before attempting the task.

ANSWER :-

PYTHON CODE :-

```
from groq import Groq
client = Groq(api_key="YOUR_GROQ_API_KEY")
prompt = """
You are an AI assistant.
If the user's request is ambiguous or incomplete, do NOT guess.
Instead, ask exactly one clarifying question before answering.
User Request:
Make this shorter.
"""
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "user", "content": prompt}
    ]
)
print(response.choices[0].message.content)
```

OUTPUT :-



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Manas/clarifying_question_groq.py =====
>>> What would you like me to make shorter?
>>> |
```

Clarifying-Question Prompt (Ambiguity Resolution) Using ChatGPT

Chosen LLM Model

- **LLM Name:** GPT-5.5
- **Provider:** OpenAI

Detailed Step-by-Step Process

Step 1: User Provides an Ambiguous Request

The user provides an incomplete or unclear instruction.

Example:

User Request:

Make this shorter.

At this stage, the request does not specify:

1. Which text should be shortened.
2. Desired length.
3. Tone to maintain.
4. Information to keep or remove.

Step 2: GPT-5.5 Receives the Input

The LLM receives the user's ambiguous request as input.

At this stage, GPT-5.5 performs:

- Text understanding
- Intent detection

- Context analysis
- Ambiguity identification

Step 3: Detect Ambiguity

GPT-5.5 analyzes whether enough information is available to complete the request.

Request	Status
Make this shorter.	Ambiguous
Rewrite this.	Ambiguous
Summarize it.	Ambiguous
Translate this to Telugu.	Clear (if text is provided)

GPT-5.5 determines that the request lacks sufficient information.

Step 4: Ask a Clarifying Question

Instead of making assumptions, GPT-5.5 asks a clear and relevant question.

Example:

Clarifying Question:

Could you please share the text you would like me to shorten?

This ensures that the assistant understands the user's actual intention before proceeding.

Step 5: User Provides Clarification

The user responds with the missing information.

Example:

Text:

Artificial Intelligence is transforming various industries by automating repetitive tasks and improving efficiency.

Instruction:

Make it about 20 words.

GPT-5.5 now has enough information to continue.

Step 6: Generate the Requested Output

GPT-5.5 performs the requested task based on the clarification.

Example Output:

Artificial Intelligence improves efficiency by automating tasks across various industries.

Step 7: Verify the Response

GPT-5.5 checks:

- Whether the requested length is achieved.
- Whether the original meaning is preserved.
- Whether the response is grammatically correct.
- Whether the user's clarification has been followed.

Step 8: Deliver the Final Response

GPT-5.5 provides the completed output only after receiving sufficient information.

Final Response:

Shortened Text:

Artificial Intelligence improves efficiency by automating tasks across many industries.

Conclusion

GPT-5.5 follows an ambiguity-resolution approach by first checking whether the user's request contains enough information. If essential details are missing, it asks a clear clarifying question instead of making assumptions. After receiving the required information, GPT-5.5 completes the task accurately and provides a clear, relevant, and user-focused response.